

Hello Farmer

AI-Powered Conversational Agricultural Voice Assistant for Ethiopian Farmers

MVP Project Documentation

1. Executive Summary

Hello Farmer is an AI-powered agricultural voice assistant designed to make agricultural information easier to access for Ethiopian farmers through natural voice conversations.

The core idea is simple:

Do not make the farmer learn how to use the agricultural system. Make the agricultural system learn how to understand the farmer.

Many Ethiopian farmers face difficulties accessing timely and reliable agricultural information. These challenges include limited access to agricultural experts, long distances to extension services, limited internet connectivity, limited smartphone access, language barriers, limited access to weather information, difficulty identifying crop diseases and pests, and limited access to personalized agricultural guidance.

Existing agricultural telephone services, including the 8028 Farmers' Hotline, provide an important foundation for agricultural information access. However, traditional IVR systems require users to navigate menus and make keypad selections. This can create significant friction, particularly for users with limited literacy or users who are unfamiliar with complex menu structures.

According to the interaction analytics provided for this project, more than 60% of first-time callers reportedly drop off before receiving agricultural advice. Premature keypad selections, multi-level menus, registration requirements, and rigid routing can cause invalid selections, errors, and early disconnections.

Hello Farmer addresses this problem by introducing a conversational AI layer.

Instead of asking a farmer to navigate multiple menus, Hello Farmer allows the farmer to simply speak naturally. The AI listens, understands the farmer's language and agricultural question,

extracts relevant context, retrieves information from a trusted agricultural knowledge base, uses weather information when necessary, generates grounded agricultural advice, and responds through voice.

For the MVP, the telecommunications component will not connect directly to a real telecom operator or the production 8028 infrastructure. Instead, **Asterisk will run in Docker as an open-source telephony server**, allowing the complete voice interaction to be demonstrated using SIP-based calls and local or development softphones.

The MVP will support five languages:

- Amharic
- Afaan Oromo
- Tigrigna
- Somali
- Sidama

The system will also provide localized post-call SMS summaries and a proactive agricultural warning concept. Since the MVP will use free-tier or open-source infrastructure, SMS delivery can initially be implemented through a development/sandbox gateway, a mock SMS provider, or a simulated notification layer. Production telecom integration can be added later.

Hello Farmer is therefore designed as a practical proof of concept demonstrating how conversational AI can transform agricultural information access without requiring farmers to learn complicated technology.

2. Background and Context

Agriculture plays a major role in Ethiopia's economy and supports a large proportion of the country's population.

However, access to agricultural knowledge is not evenly distributed.

A farmer may need information about:

- What crop to plant
- When to plant
- Fertilizer application
- Pest management
- Disease symptoms
- Weather conditions
- Rainfall

- Crop growth stages
- Harvest timing
- Soil management
- Production risks
- Expected yield
- Agricultural finance
- Climate-related risks

In many cases, the information exists but is difficult to access at the exact moment the farmer needs it.

The problem is therefore not only the availability of agricultural knowledge.

It is also the accessibility, discoverability, language, timing, and personalization of that knowledge.

A farmer should not have to understand an agricultural information system before receiving agricultural advice.

Hello Farmer proposes a conversational interface that allows the farmer to communicate using ordinary speech.

3. Problem Statement

3.1 Limited Access to Agricultural Experts

Many farmers have limited access to agricultural experts and extension services.

Farmers may live far from agricultural offices or extension centers. Getting professional advice may require transportation, time, and additional cost.

As a result, farmers may rely on:

- Informal advice
- Previous experience
- Neighbors
- Unverified information
- Delayed agricultural support

This can be particularly harmful when a problem requires immediate action.

3.2 Limited Internet and Smartphone Access

A large proportion of rural farmers cannot depend on smartphones or continuous internet connectivity.

A solution that requires:

- Mobile applications
- Continuous internet access
- Large downloads
- High-end smartphones
- Digital literacy

cannot be the only agricultural information channel.

Voice through an ordinary telephone is therefore an important accessibility layer.

3.3 Language Barriers

Agricultural information is often distributed in limited languages.

Farmers may be more comfortable communicating in their local language than in English or another widely used language.

Hello Farmer therefore focuses on multilingual conversational interaction.

The MVP targets:

- Amharic
 - Afaan Oromo
 - Tigrigna
 - Somali
 - Sidama
-

3.4 Difficulty Accessing Weather Information

Agricultural decisions are strongly influenced by weather.

Farmers may need to know:

- Whether rain is expected

- Whether heavy rainfall is likely
- Whether spraying conditions are suitable
- Whether extreme temperatures are expected
- Whether weather conditions create crop risks

Weather information becomes significantly more useful when connected to the farmer's crop and location.

3.5 Crop Disease and Pest Problems

Farmers frequently need help understanding crop problems.

For example:

"My maize leaves are becoming yellow. What should I do?"

Answering such questions effectively requires understanding:

- Crop
- Growth stage
- Symptoms
- Affected plant parts
- Weather
- Spread pattern
- Farming practices
- Possible pests
- Possible diseases

A generic answer may not be sufficient.

Hello Farmer therefore combines conversational questioning with agricultural knowledge retrieval.

3.6 Lack of Personalized Agricultural Guidance

Traditional agricultural information is often static.

For example, a document may explain how maize should be managed, but the farmer may actually need advice based on:

- Location

- Crop
- Growth stage
- Planting date
- Farm size
- Current weather
- Current problem

Hello Farmer attempts to transform general agricultural knowledge into context-aware guidance.

4. The 8028 IVR Problem

The 8028 Farmers' Hotline provides an important telephone-based agricultural information service.

However, traditional IVR systems depend heavily on menu navigation and keypad interaction.

A typical interaction may require a farmer to:

1. Call the service.
2. Listen to a menu.
3. Press a number.
4. Listen to another menu.
5. Press another number.
6. Select a crop.
7. Provide location information.
8. Continue through additional menu levels.
9. Finally reach agricultural information.

This interaction model creates several problems.

Menu Fatigue

Farmers must listen to multiple options before reaching the relevant service.

Literacy and Cognitive Constraints

Users may have difficulty remembering menu structures, codes, or long sequences.

Premature Keypresses

Some callers press keys before all options have been presented.

This can result in:

- Invalid routing
- Incorrect selections
- System errors
- Premature disconnections

Registration Friction

Rigid registration processes may require precise numeric inputs such as:

- District codes
- Woreda codes
- Crop identifiers

This creates friction before the farmer receives useful information.

High First-Time Caller Attrition

According to the interaction analytics provided for this project, more than 60% of first-time callers reportedly drop off before receiving agricultural advice.

This indicates that the problem is not simply the availability of a telephone service.

The interaction model itself can become a barrier.

5. Core Insight

The central problem can be summarized as:

Farmers are being asked to adapt to the agricultural information system instead of allowing the system to understand the farmer naturally.

Hello Farmer changes this interaction model.

Instead of:

Caller → Menu → Keypad → Menu → Keypad → Information

Hello Farmer provides:

Caller → Natural Speech → AI Understanding → Agricultural Advice

6. Proposed Solution

Hello Farmer introduces an AI voice layer that can operate alongside existing telecommunications and agricultural information infrastructure.

The goal is not simply to replace existing services.

The goal is to extend their reach and capacity through conversational AI.

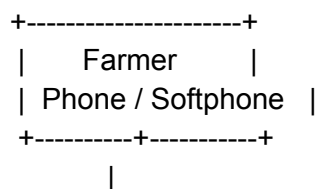
The MVP will demonstrate this concept using:

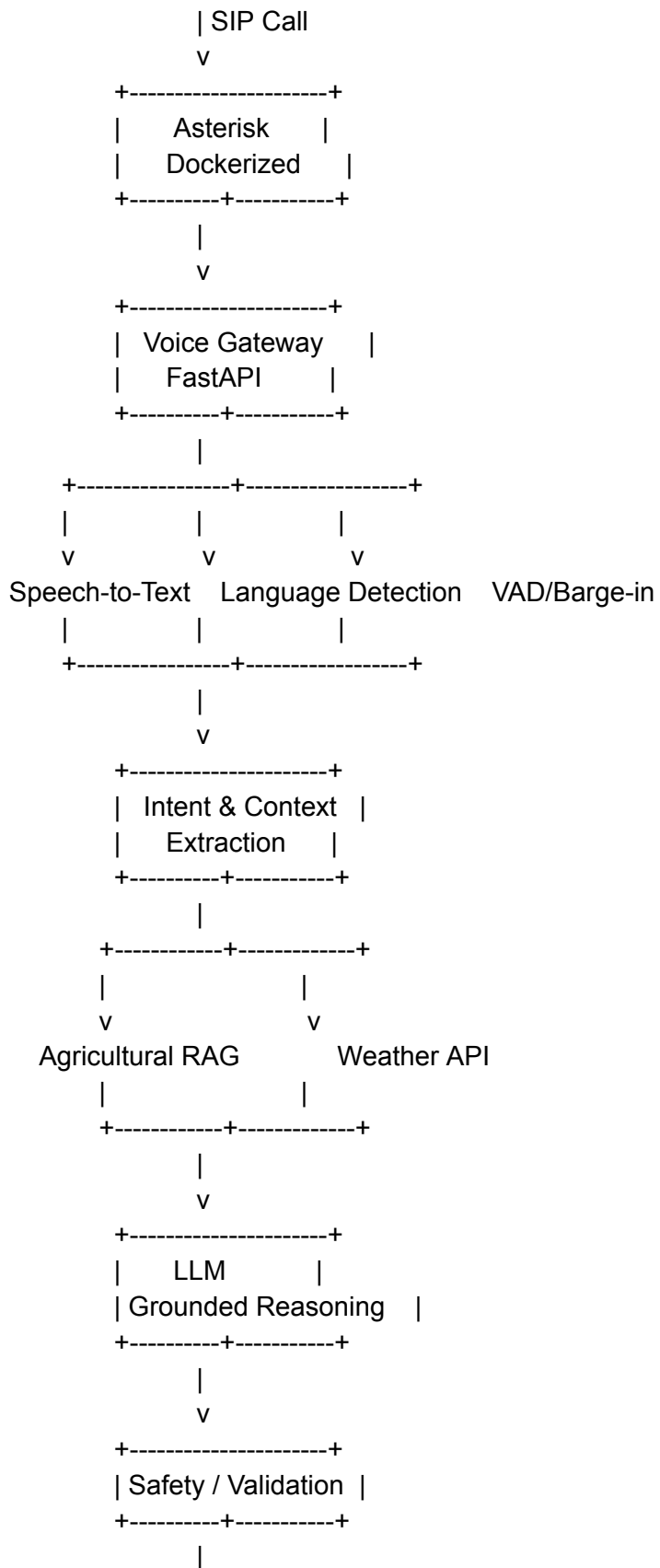
- Asterisk
- Docker
- SIP
- Softphone
- Speech-to-Text
- Language Detection
- LLM
- Agricultural RAG
- Weather API
- Text-to-Speech
- PostgreSQL
- pgvector
- Redis
- FastAPI
- React or Next.js

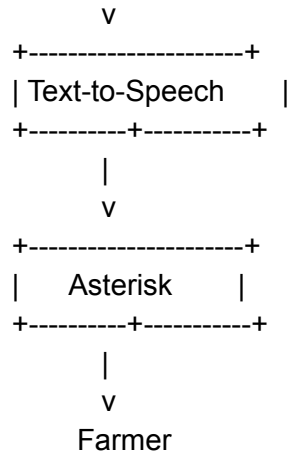
No production telecom connection is required for the MVP.

7. MVP Architecture

The initial architecture will use Asterisk as the telephony layer.







8. Why Asterisk for the MVP?

The MVP does not require a telecom operator integration.

Asterisk provides an open-source telephony environment that can simulate the telephone experience.

Asterisk will run inside Docker.

The developer can use:

- SIP softphones
- Local SIP clients
- Development phones
- Test extensions

This allows the project to demonstrate:

- Incoming calls
- Call routing
- Audio streaming
- AI responses
- Conversation handling
- Call termination

without paying for telecom infrastructure.

The architecture can later be connected to a SIP trunk or telecom provider.

The important point is that the AI layer remains independent from the telecom provider.

9. Docker-Based MVP Infrastructure

The MVP will use Docker Compose to simplify deployment.

Conceptually:

Docker Compose

|

+-- Asterisk

|

+-- FastAPI Backend

|

+-- PostgreSQL

|

+-- pgvector

|

+-- Redis

|

+-- Web Application

|

+-- Worker / Background Jobs

|

+-- Optional Monitoring

This allows the entire MVP to be started using a single Docker Compose environment.

The objective is to minimize infrastructure cost and operational complexity.

10. Zero-Touch Conversational AI Voice Agent

The main feature of Hello Farmer is the zero-touch conversational voice agent.

The farmer should not have to press keys to describe the problem.

After the call connects, the AI can say:

"Hello. How can I help you with your farming today?"

The farmer can respond naturally.

For example:

"My maize leaves have started turning yellow after the recent rain."

The AI then identifies:

- Crop: Maize
- Problem: Yellow leaves
- Weather context: Recent rainfall
- Potential need: Disease/nutrient/pest investigation

The AI can ask:

"How old is the maize, and are the yellow leaves appearing on the older leaves or the newer leaves?"

This creates a natural conversation instead of a menu-based interaction.

11. Zero-Keypress Interaction

The MVP eliminates DTMF-based agricultural navigation.

Traditional:

Press 1 for crop information.

Press 2 for weather.

Press 3 for disease information.

Press 4 for fertilizer.

Hello Farmer:

Farmer:

"My tomato plants are getting spots on the leaves."

AI:

"Can you tell me whether the spots are dark, yellow,

or surrounded by a yellow area?"

The system determines the user's intent automatically.

Possible intents include:

- Agricultural advice
 - Disease support
 - Pest support
 - Weather
 - Crop management
 - Yield information
 - Finance information
 - Warning explanation
-

12. Voice Activity Detection and Barge-In

Natural conversations require the farmer to be able to interrupt the AI.

For example:

AI:

"The recommended treatment is..."

Farmer:

"Wait, I want to ask..."

AI:

"Yes, please go ahead."

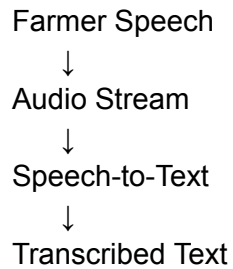
Voice Activity Detection will identify when the farmer begins speaking.

Barge-in functionality allows the AI response to stop and listen.

This prevents long automated responses from creating frustration.

13. Speech-to-Text

The voice pipeline converts spoken language into text.



The system should support the five MVP languages:

- Amharic
- Afaan Oromo
- Tigrigna
- Somali
- Sidama

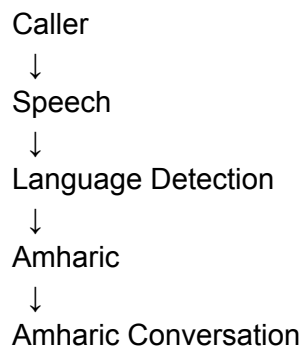
Speech recognition quality will be evaluated separately for each language.

The MVP should prioritize practical conversational accuracy rather than attempting to solve all dialect and accent variations immediately.

14. Language Detection

The system should identify the language used by the farmer.

Example:



If the language is already known from previous interactions, the system can use the stored preferred language as a strong signal.

The language preference can also be updated when the farmer changes language.

15. Five-Language MVP

The first version will support:

1. Amharic
2. Afaan Oromo
3. Tigrigna
4. Somali
5. Sidama

Language support applies to:

- Voice conversations
- AI responses
- Agricultural recommendations
- Post-call SMS summaries
- Proactive warning SMS
- Web notifications
- Relevant web content

The project will evaluate language quality separately instead of assuming that performance in one language represents performance in all five.

16. Dynamic Farmer Context Extraction

Hello Farmer will avoid requiring a long registration process before providing assistance.

Instead, the AI can extract information naturally during conversation.

For example:

Farmer:

"I planted maize two months ago on my farm near Adama."

The system can extract:

Crop: Maize

Planting time: Approximately 2 months ago

Location: Adama

Another conversation may provide:

Farm size: 2 hectares

Crop: Maize

Growth stage: Vegetative

Problem: Yellow leaves

Recent weather: Heavy rain

This information can be stored as farmer context when appropriate.

17. Farmer Context

The system can maintain useful agricultural context.

Conceptually:

Farmer

|

+-- Phone Number

+-- Preferred Language

+-- Location

+-- Crops

+-- Farm Size

+-- Planting Information

+-- Notification Preferences

Conversation-specific context:

Current Crop: Maize

Growth Stage: Vegetative

Problem: Yellow Leaves

Recent Weather: Heavy Rain

There are two types of memory:

Short-Term Conversation Memory

Information required to understand the current call.

Longer-Term Agricultural Context

Useful information retained for future conversations where appropriate and with suitable privacy controls.

The system should not store unnecessary personal information.

18. Agricultural RAG

The agricultural knowledge system will use Retrieval-Augmented Generation.

The purpose of RAG is to ensure that agricultural answers are grounded in trusted information rather than relying entirely on the LLM's internal knowledge.

Architecture:

Agricultural Documents



Document Processing



Cleaning



Chunking



Metadata Extraction



Embeddings



Vector Database



Semantic Search



Relevant Knowledge



LLM



19. Agricultural Knowledge Sources

The knowledge base can include:

- Ethiopian agricultural manuals
- Government publications
- Agricultural extension materials
- Crop production guides
- Crop disease guides
- Pest management guides
- Fertilizer guidance
- Soil management materials
- Climate guidance
- Agricultural research publications
- Research institution materials
- Trusted international agricultural resources

For the MVP, only legally usable and trustworthy documents should be included.

20. RAG Metadata

Each document chunk should contain useful metadata.

Example:

```
document_id
title
source
crop
region
language
topic
publication_date
page
chunk_text
embedding
```

This allows the retrieval system to improve relevance.

For example, a farmer asking about maize in a particular region should receive information relevant to maize rather than unrelated crop information.

21. Vector Search

The MVP can use PostgreSQL with pgvector.

This keeps the infrastructure simple.

Instead of deploying many separate databases:

PostgreSQL

|

- +-- Farmer Data
- +-- Conversation Data
- +-- Agricultural Documents
- +-- Knowledge Chunks
- +-- Vector Embeddings
- +-- Weather Records
- +-- Warning Records

pgvector provides vector similarity search inside PostgreSQL.

FAISS or Qdrant can be evaluated later if retrieval requirements become more complex.

22. Personalized Agricultural Advisory

The AI should combine retrieved agricultural knowledge with farmer context.

Example:

Farmer:

"My maize is two months old and the leaves are yellow."

System Context:

Crop = Maize

Age = 2 months

Location = Known

Weather = Recent heavy rainfall

RAG:

Retrieves maize yellowing guidance.

LLM:

Combines the context and retrieved information.

Response:

Provides possible causes and recommended next steps.

The AI should ask additional questions when the information is insufficient.

It should not immediately make unsupported conclusions.

23. Crop Disease and Pest Support

The system will support conversational disease and pest investigation.

It can ask about:

- Crop type
- Plant age
- Symptoms
- Affected parts
- Spread pattern
- Weather
- Pest observations
- Farming practices

Example:

Farmer:

"My tomato leaves have spots."

AI:

"When did you first notice the spots?"

Farmer:

"About one week ago."

AI:

"Are the spots dark, yellow, or surrounded by yellow areas?"

The system should communicate uncertainty.

It should not claim:

"Your crop definitely has disease X."

Instead:

"These symptoms may be consistent with several problems. Based on the available information, one possibility is X. The following observations can help distinguish it from other causes."

Pesticide recommendations should be especially cautious and grounded in trusted sources.

24. Weather and Climate Intelligence

Weather information will be integrated when relevant.

The system may retrieve:

- Rainfall forecast
- Temperature
- Wind
- Heavy rainfall risk
- Extreme temperature
- Other available weather indicators

Example:

Farmer:

"Can I spray my crop tomorrow?"

The system can use:

- Farmer location
- Crop
- Relevant agricultural guidance
- Weather forecast

and provide a practical answer.

Weather data should not be treated as perfectly accurate.

25. Yield-Related Insights

The MVP can provide basic yield-related insights.

Potential inputs include:

- Crop
- Farm size
- Location
- Planting date
- Weather
- Soil information where available
- Farming practices
- Historical yield
- Growth information

The system should communicate that yield predictions are estimates.

Example:

"Based on the information provided, the estimated yield range is..."

rather than presenting a prediction as guaranteed.

A more advanced yield prediction model can be introduced later after collecting sufficient local data.

26. Agricultural Finance Support

Hello Farmer can provide agricultural finance-related information and basic risk insights.

Potential factors include:

- Farm size
- Crop
- Production history
- Expected yield
- Farming activity
- Weather risk
- Production risk

The system should not make final lending decisions.

Instead, it can provide information and structured agricultural risk indicators.

Example:

Farm Information



Production Context



Weather Risk



Agricultural Risk Indicators



Information for Farmer / Financial Workflow

27. Proactive Agricultural Warning System

Hello Farmer should not only respond when farmers call.

It can also provide proactive warnings.

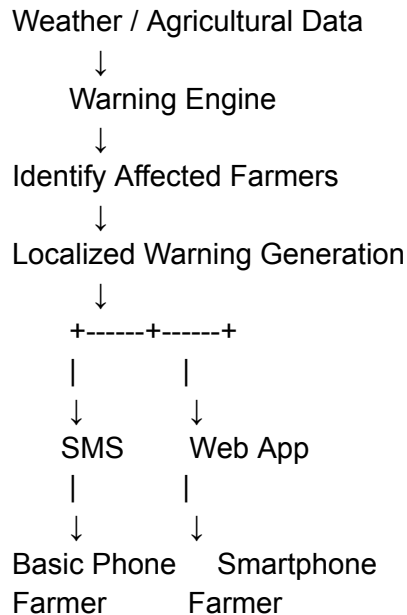
Examples include:

- Heavy rainfall
- Flood risk
- Drought
- Extreme temperature
- Strong wind
- Frost risk
- Pest outbreak risk
- Crop disease risk

- Important crop-management warnings

The warning engine identifies relevant risks and sends notifications to registered farmers.

28. Warning Architecture



The warning engine determines:

1. What is happening?
 2. Where is it happening?
 3. Which farmers may be affected?
 4. How severe is the risk?
 5. What action should be recommended?
 6. Which notification channels should be used?
-

29. Warning Targeting

Warning targeting can consider:

- Farmer location
- Farm location

- Crops
- Warning type
- Severity
- Time window
- Notification preferences

For example:

Heavy Rain Warning



Affected Region



Farmers in Region



Farmers Growing Relevant Crops



Localized Warning



SMS / Web Notification

30. Warning Severity

The MVP can use:

LOW

MEDIUM

HIGH

CRITICAL

The exact severity rules should eventually be validated by agricultural experts.

The system should avoid generating excessive warnings because too many notifications can cause farmers to ignore future alerts.

31. Post-Call SMS Summary

After the farmer finishes a call, Hello Farmer can generate a concise SMS summary.

The SMS is not a transcript.

It should summarize the important information.

Example:

Hello Farmer Summary:

Crop: Maize

Problem:

Yellow leaves after recent rainfall.

Advice:

Check whether the affected leaves are old or new and inspect the plants for signs of pests or disease.

Follow-up:

You can call Hello Farmer again if the symptoms continue.

The actual SMS should be much shorter because of SMS length limitations.

The summary should be generated directly in the farmer's language whenever possible.

32. Two Types of SMS

Hello Farmer will distinguish between two different SMS workflows.

Post-Call SMS

Triggered after a farmer completes a call.

Purpose:

- Remember the advice
- Provide recommended actions
- Highlight warnings
- Provide follow-up information

Proactive Warning SMS

Triggered automatically when a warning event affects registered farmers.

Purpose:

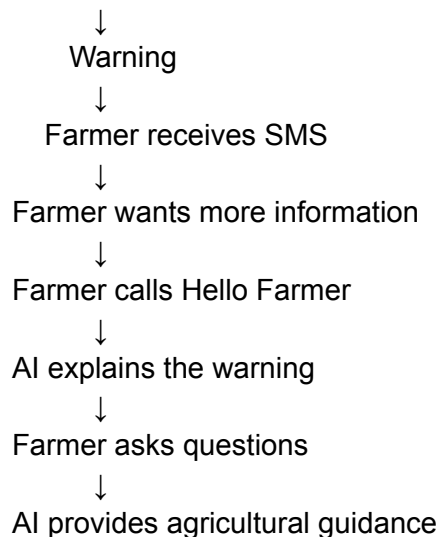
- Inform farmers about risks
- Provide recommended action
- Encourage further inquiry when needed

These are separate workflows.

33. Warning Follow-Up Loop

An important feature of Hello Farmer is the connection between proactive warnings and conversational assistance.

Weather / Agricultural Risk



This creates a:

Detect → Warn → Explain → Advise

loop.

34. Modern and Commercial Farmer Web Experience

Hello Farmer will provide a richer web experience for farmers who have smartphones and internet access.

The web experience complements the voice system.

It is not intended to replace voice.

The web interface can provide:

- Agricultural information
- Warning notifications
- Weather information
- Conversation summaries
- Agricultural recommendations
- Relevant crop information
- Notification history
- Basic agricultural insights

The MVP should keep the web interface focused rather than attempting to build a complete farm-management platform.

35. One Platform, Two Experiences

Hello Farmer is one platform serving different technology environments.

Rural / Smallholder Experience

Basic Phone



Voice



AI Agricultural Assistant



SMS

The farmer does not need:

- Smartphone
- Mobile application
- Internet data

for the core voice interaction.

Modern / Commercial Farmer Experience

Smartphone

↓

Web Application

+

Voice

+

SMS

This provides a richer experience for farmers who have access to smartphones and internet connectivity.

The same AI intelligence layer supports both experiences.

36. Important Connectivity Principle

The system should be described accurately as:

Internet-independent for the farmer, not completely offline for the backend.

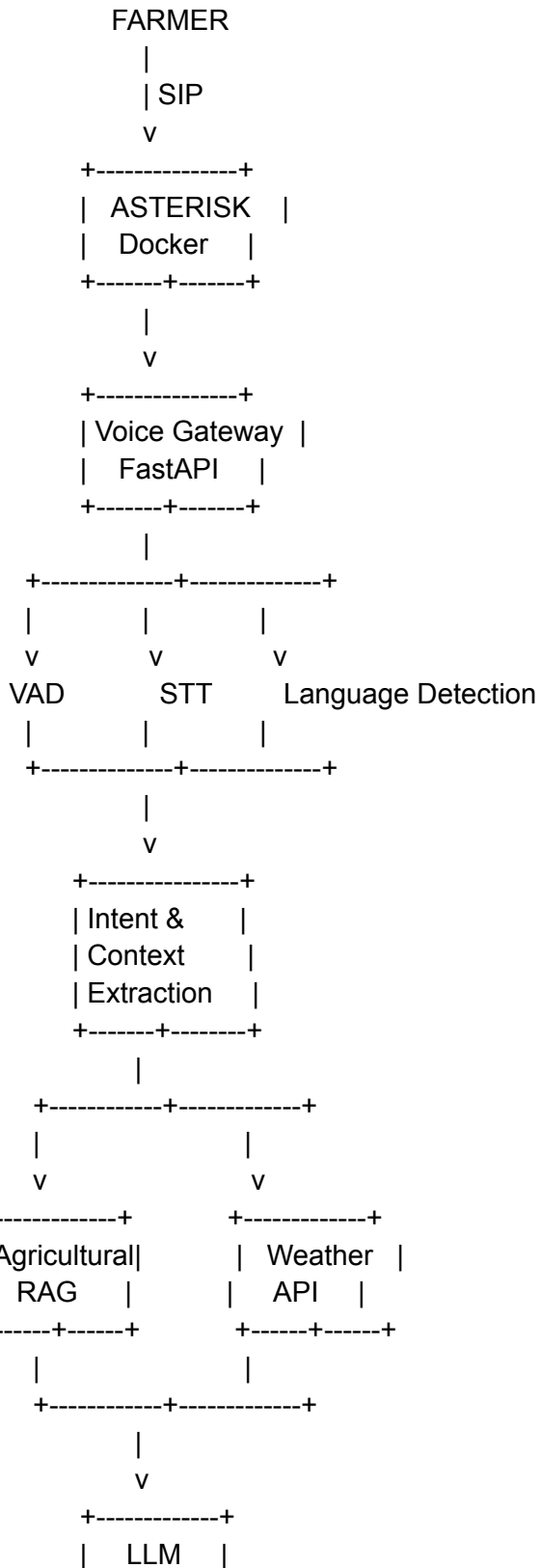
A farmer using the telephone does not need mobile internet data for the core voice interaction.

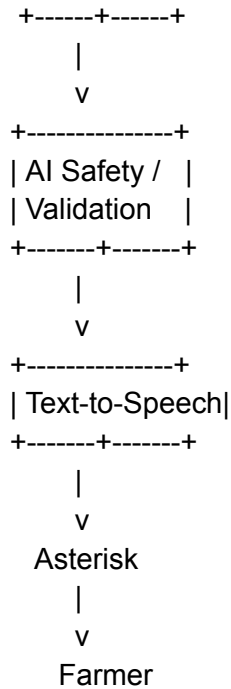
However, the backend infrastructure requires connectivity for:

- AI services
- Weather APIs
- Databases
- RAG
- External services

The MVP therefore focuses on reducing the connectivity requirements on the farmer side.

37. Technical Architecture





38. Backend Services

The MVP backend can be organized into logical services.

FastAPI Backend

- |
- +-- Voice Service
- |
- +-- Conversation Service
- |
- +-- Language Service
- |
- +-- Intent Service
- |
- +-- Farmer Context Service
- |
- +-- RAG Service
- |
- +-- Weather Service
- |
- +-- Warning Service
- |

+-- SMS Service
|
+-- Notification Service
|
+-- AI Safety Service
|
+-- Analytics Service

These do not necessarily need to be separate microservices in the MVP.

A modular monolith is preferable initially.

This reduces development time and infrastructure complexity.

39. Recommended MVP Architecture Style

The project should avoid unnecessary microservices during the MVP.

Recommended:

Docker Compose
|
+-- Asterisk
|
+-- FastAPI Modular Backend
|
+-- PostgreSQL + pgvector
|
+-- Redis
|
+-- Web App
|
+-- Background Worker

This is easier to:

- Develop
- Test
- Debug
- Deploy

- Maintain

Microservices can be introduced later if real traffic requires them.

40. Database Design

Potential MVP tables include:

farmers
farmer_context
farmer_locations
farmer_crops
calls
conversation_messages
call_transcripts
language_sessions
agricultural_documents
knowledge_chunks
weather_records
warnings
farmer_warnings
sms_messages
web_notifications
ai_interactions

41. Farmer Context Model

Conceptually:

farmers

id
phone_number
preferred_language
created_at
updated_at

farmer_context

```
-----  
id  
farmer_id  
farm_size  
location  
additional_context  
updated_at
```

```
farmer_crops
```

```
-----  
id  
farmer_id  
crop  
planting_date  
growth_stage  
created_at  
updated_at
```

The actual database schema can be simplified for the MVP.

42. Warning Data Model

Example:

```
warnings  
-----  
warning_id  
type  
severity  
location  
crop  
title  
description  
recommended_action  
source  
created_at  
expires_at
```

Notification tracking:

farmer_warnings

notification_id

warning_id

farmer_id

channel

language

status

sent_at

delivered_at

43. Conversation Data

A call can be represented as:

calls

id

farmer_id

session_id

started_at

ended_at

duration

language

status

Messages:

conversation_messages

id

call_id

role

content

timestamp

Possible roles:

farmer

assistant
system

44. SMS Data

sms_messages

id
farmer_id
call_id
warning_id
language
message
status
provider
sent_at
delivered_at

For the MVP, the provider can be:

mock
sandbox
development

This allows the full SMS workflow to be demonstrated without requiring a paid production SMS provider.

45. Technology Stack

Backend

Python

API

FastAPI

Database

PostgreSQL

Vector Search

pgvector

Cache

Redis

Telephony

Asterisk

Containerization

Docker / Docker Compose

Frontend

React or Next.js

AI

Multilingual LLM

Speech-to-Text

Multilingual STT model/service

Text-to-Speech

Multilingual TTS model/service

RAG

Embeddings + pgvector

Weather

Free-tier or open weather API

SMS

Development/sandbox/mock SMS gateway for MVP

46. Free-Tier MVP Strategy

The project is intentionally designed around free and open-source infrastructure.

The goal is to avoid paid infrastructure during the prototype stage.

Potential approach:

Asterisk

→ Open Source / Local

Docker

→ Open Source / Local

FastAPI

→ Open Source / Local

PostgreSQL

→ Open Source / Local

pgvector

→ Open Source / Local

Redis

→ Open Source / Local

React / Next.js

→ Open Source / Local

RAG

→ Open Source

Development SMS

→ Mock / Sandbox

Weather

→ Free Tier

AI Models

→ Free Tier or locally available models where practical

The exact AI provider should be selected based on:

- Free-tier availability
- Language quality
- Latency
- API limits
- Cost after the free tier
- Reliability
- Privacy

The MVP should not depend on a paid telecom provider.

47. Local Development Environment

The complete MVP should be runnable locally.

Example:

Developer Machine

|

+-- Docker

|

+-- Asterisk

+-- FastAPI

+-- PostgreSQL

+-- pgvector

+-- Redis

+-- Web App

+-- Worker

A SIP softphone connects to Asterisk.

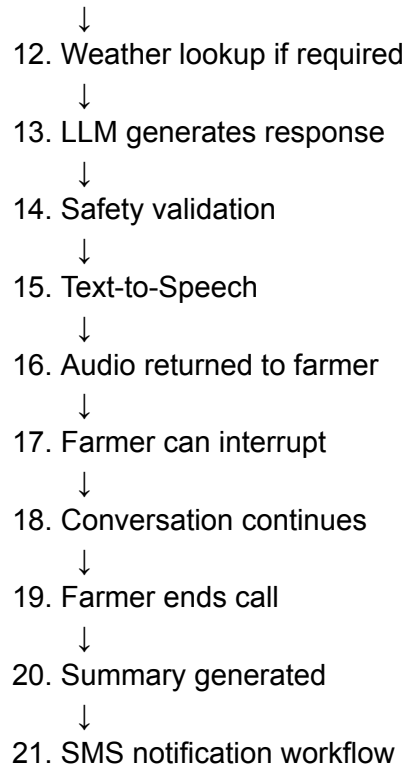
Example:



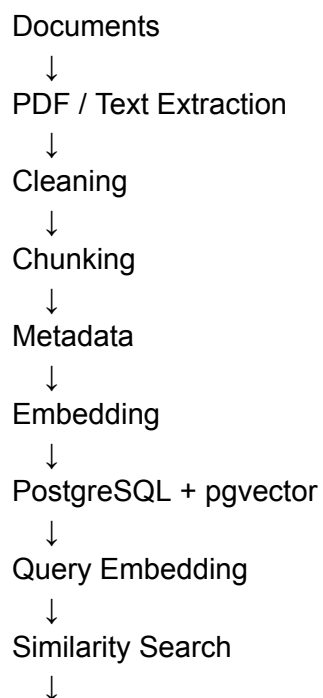
This makes it possible to demonstrate the system without a real telephone number.

48. Voice Call Flow

1. Farmer calls test extension
- ↓
2. Asterisk receives call
- ↓
3. Asterisk routes call to Hello Farmer
- ↓
4. AI greeting
- ↓
5. Farmer speaks
- ↓
6. Audio captured
- ↓
7. Speech-to-Text
- ↓
8. Language Detection
- ↓
9. Intent Detection
- ↓
10. Context Extraction
- ↓
11. RAG Retrieval



49. Agricultural RAG Pipeline



Top Relevant Chunks



LLM Context



Grounded Answer

The retrieval system should prioritize relevant agricultural information before the LLM generates its answer.

50. AI Response Generation

The LLM should receive:

System Instructions

+

Farmer Question

+

Conversation Context

+

Farmer Agricultural Context

+

Weather Data

+

Retrieved Agricultural Knowledge

Conceptually:

Question

+

Context

+

RAG

+

Weather



LLM



Safety Validation



51. AI Safety

Agricultural recommendations can affect farmers' livelihoods.

Therefore, the system must prioritize safety.

The AI should:

- Prefer trusted agricultural sources
- Ground answers in retrieved information
- Communicate uncertainty
- Avoid unsupported pesticide recommendations
- Avoid inventing fertilizer doses
- Consider crop and location
- Consider growth stage
- Consider weather
- Avoid making unsupported disease diagnoses
- Avoid presenting predictions as guarantees

If reliable information is unavailable, the system should clearly state the limitation instead of inventing an answer.

52. Privacy and Data Protection

The MVP may process:

- Phone numbers
- Voice recordings
- Transcripts
- Farmer location
- Crop information
- Agricultural context
- Notification preferences

The system should collect only information required for the service.

Privacy policies should define:

- What is collected
- Why it is collected
- How long it is stored
- Who can access it
- How it is protected
- When it is deleted

Voice recordings and transcripts should not be retained indefinitely without a clear reason.

53. Security

The backend should implement:

- Authentication
- Authorization
- Role-based access control
- Encryption
- Secure API communication
- Input validation
- Rate limiting
- Secret management
- Audit logs
- Database access controls

Environment variables should be used for:

API keys

Database credentials

AI provider keys

Weather API keys

SMS credentials

JWT secrets

Secrets should not be committed to GitHub.

54. Testing Strategy

Voice Testing

Test:

- Background noise
 - Different accents
 - Speaking speeds
 - Poor audio quality
 - Long pauses
 - Interruptions
 - Barge-in
 - 8 kHz telephony audio
 - Short questions
 - Long questions
-

55. Language Testing

Each language should have its own evaluation set.

Amharic
Afaan Oromo
Tigrigna
Somali
Sidama

Metrics can include:

- Speech recognition accuracy
 - Language detection accuracy
 - Response correctness
 - Naturalness
 - Agricultural terminology
 - Translation quality
 - TTS clarity
-

56. Agricultural Testing

Agricultural experts or trusted agricultural references should be used to evaluate:

- Accuracy
- Safety
- Relevance
- Local applicability
- Completeness
- Uncertainty communication

Disease and pesticide-related responses require particular attention.

57. Warning Testing

The warning engine should be tested for:

- Correct affected locations
- Correct farmer targeting
- Correct severity
- Correct crop targeting
- Correct language
- Correct timing
- SMS generation
- Web notification generation

False warnings should be monitored because excessive false alerts can reduce trust.

58. SMS Testing

Test:

- Summary generation
- Language correctness
- Message length
- Delivery workflow
- Warning SMS

- Post-call SMS
- Duplicate prevention

For the free-tier MVP, delivery can initially be simulated through a mock SMS service.

59. Monitoring and Analytics

Voice metrics:

- Number of calls
- Successful calls
- Average call duration
- Call completion
- Response latency
- Dropped calls
- First-time caller drop-off
- Invalid routing rate

AI metrics:

- Language detection accuracy
- Speech recognition accuracy
- Intent accuracy
- Retrieval accuracy
- Answer quality
- Hallucination rate

Language metrics should be tracked separately:

Amharic

Afaan Oromo

Tigrigna

Somali

Sidama

Warning metrics:

- Warnings generated
- Farmers affected
- SMS delivery
- Web delivery

- Warning accuracy
- False warning rate

SMS metrics:

- Summary generation success
 - Delivery rate
 - Correct language
 - Summary quality
-

60. Key MVP Success Metrics

The project should measure improvement against the traditional IVR interaction model.

Important metrics include:

Accessibility

Percentage of callers who successfully reach an agricultural answer.

First-Time Caller Completion

Percentage of first-time callers who complete a meaningful interaction.

Menu Abandonment

Reduction in menu-related abandonment compared with the existing IVR baseline.

AI Response Latency

Time between farmer speech and AI response.

Language Accuracy

Performance across all five MVP languages.

Retrieval Quality

Whether the system retrieves relevant agricultural information.

Answer Safety

Percentage of evaluated responses that meet agricultural safety criteria.

Warning Accuracy

Percentage of warnings correctly targeted to affected locations and farmers.

61. MVP Scope

The MVP will include:

Voice

- Asterisk
- Dockerized telephony
- SIP-based calls
- Zero-keypress interaction
- Speech-to-text
- Language detection
- Intent detection
- Context extraction
- Conversation memory
- Text-to-speech
- Voice Activity Detection
- Barge-in

Languages

- Amharic
- Afaan Oromo
- Tigrigna
- Somali
- Sidama

Agricultural Intelligence

- Agricultural RAG
- Personalized agricultural advisory
- Crop disease and pest support
- Weather information

- Crop management guidance
- Basic yield-related insights
- Agricultural finance information

Notifications

- Post-call SMS summaries
- Proactive agricultural warnings
- SMS warning workflow
- Web warning notifications

Web

- Basic modern farmer web experience
- Warning notifications
- Agricultural information
- Conversation summaries

Infrastructure

- Docker
 - Asterisk
 - FastAPI
 - PostgreSQL
 - pgvector
 - Redis
 - React/Next.js
-

62. Explicitly Out of Scope for MVP

The following are intentionally excluded:

- SACCO functionality
- Standalone Farmer Account product
- Human escalation or human handoff
- Irrigation Scheduler
- Production telecom integration
- Direct 8028 production integration
- Paid telecom infrastructure

- Large-scale production deployment
- Complex farm management ERP
- Full financial lending platform
- Fully automated lending decisions
- Advanced autonomous agricultural decisions

These features can be considered in future versions if required.

63. MVP Development Phases

Phase 1: Voice Prototype

Build:

- Docker environment
- Asterisk
- SIP extension
- Softphone
- Basic call routing
- FastAPI voice backend

Goal:

A user can make a call and receive an AI-generated voice response.

Phase 2: Conversational Voice

Add:

- Speech-to-text
- Text-to-speech
- Conversation loop
- Language detection
- Basic intent detection

Goal:

The farmer can have a natural conversation with the AI.

Phase 3: Five-Language Support

Add:

- Amharic
- Afaan Oromo
- Tigrigna
- Somali
- Sidama

Goal:

The same conversational system works across the five MVP languages.

Phase 4: Dynamic Context and Barge-In

Add:

- Farmer context extraction
- Conversation memory
- VAD
- Barge-in
- Better turn detection

Goal:

The conversation becomes more natural and personalized.

Phase 5: Agricultural RAG

Build:

- Document ingestion
- PDF processing
- Chunking
- Embeddings
- pgvector
- Semantic search

- RAG prompt construction

Goal:

The AI provides grounded agricultural information.

Phase 6: Weather Integration

Add:

- Weather API
- Location lookup
- Weather context
- Agricultural weather reasoning

Goal:

Weather becomes part of agricultural decision support.

Phase 7: Post-Call SMS

Add:

- Conversation summarization
- Localized summary generation
- SMS message generation
- Mock or sandbox SMS provider

Goal:

Farmers receive useful information after the call.

Phase 8: Warning Engine

Add:

- Weather risk detection
- Agricultural warning rules

- Severity
- Location targeting
- Farmer targeting

Goal:

The system can proactively identify agricultural risks.

Phase 9: Warning Notifications

Add:

- Warning SMS
- Web notifications
- Notification tracking
- Language localization

Goal:

Registered farmers receive relevant warnings.

Phase 10: Web Experience

Add:

- React/Next.js interface
- Warning dashboard
- Agricultural information
- Conversation summaries
- Basic farmer-facing features

Goal:

Smartphone users receive a richer experience.

Phase 11: Evaluation and Pilot

Evaluate:

- Voice quality
- Language performance
- Agricultural accuracy
- RAG quality
- Warning accuracy
- Latency
- User experience

Then conduct a controlled MVP pilot.

64. Recommended MVP Project Structure

```
hello-farmer/
|
+-- backend/
| |
| | +-- app/
| | | +-- api/
| | | +-- voice/
| | | +-- conversation/
| | | +-- language/
| | | +-- intent/
| | | +-- farmer_context/
| | | +-- rag/
| | | +-- weather/
| | | +-- warnings/
| | | +-- sms/
| | | +-- notifications/
| | | +-- safety/
| | | +-- analytics/
| | | +-- database/
| | | +-- main.py
| |
| | +-- tests/
| | +-- requirements.txt
|
+-- asterisk/
| +-- config/
| +-- extensions/
| +-- sounds/
|
```

```
+-- frontend/  
| +-- app/  
| +-- components/  
| +-- services/  
|  
+-- rag/  
| +-- documents/  
| +-- ingestion/  
| +-- evaluation/  
|  
+-- workers/  
|  
+-- docker/  
|  
+-- docker-compose.yml  
|  
+-- .env.example  
|  
+-- README.md
```

65. API Structure

Potential FastAPI endpoints:

```
POST /voice/session  
POST /voice/audio  
POST /conversation/message  
POST /conversation/end
```

```
GET /farmer/context  
POST /farmer/context
```

```
POST /rag/search  
POST /rag/ingest
```

```
GET /weather  
GET /weather/risk
```

```
GET /warnings  
POST /warnings/evaluate
```


POST /notifications/sms
POST /notifications/web

GET /analytics/calls
GET /analytics/ai
GET /analytics/warnings

The exact API structure can be simplified during implementation.

66. Background Jobs

Redis can support asynchronous background processing.

Examples:

Call Ends
↓
Queue
↓
Generate Summary
↓
Generate SMS
↓
Send / Simulate SMS

Warning:

Weather Data
↓
Queue
↓
Warning Evaluation
↓
Affected Farmers
↓
Generate Notifications

This prevents slow background tasks from blocking the voice interaction.

67. Latency Optimization

Voice systems require low latency.

The MVP should optimize:

Farmer Speech

↓

STT

↓

Intent / Context

↓

RAG

↓

LLM

↓

TTS

↓

Audio

Possible optimizations include:

- Streaming audio
- Streaming STT
- Smaller models where appropriate
- Cached embeddings
- Fast vector retrieval
- Redis caching
- Short prompts
- Limited RAG context
- Streaming TTS
- Asynchronous processing

A sub-second response target can be treated as an engineering goal where technically feasible, rather than a guaranteed MVP requirement.

68. Cost Optimization

Because the MVP is designed around free tiers, cost should be controlled from the beginning.

Strategies:

- Run Asterisk locally
- Use Docker
- Use PostgreSQL locally
- Use pgvector
- Use Redis locally
- Use free AI tiers where available
- Use open-source models where practical
- Cache repeated requests
- Limit unnecessary LLM calls
- Limit RAG context
- Use mock SMS during development
- Use free weather APIs
- Avoid production telecom integration

The MVP should prove the product concept before introducing paid infrastructure.

69. Risks and Mitigation

Risk 1: Speech Recognition Errors

Local languages and agricultural terminology may be difficult for generic STT systems.

Mitigation:

- Language-specific evaluation
 - Agricultural vocabulary testing
 - Multiple model/provider evaluation
 - Human-reviewed test datasets
-

Risk 2: LLM Hallucination

The AI may produce incorrect agricultural advice.

Mitigation:

- RAG
 - Trusted sources
 - Response validation
 - Agricultural evaluation
 - Uncertainty communication
-

Risk 3: Poor TTS Quality

Some languages may have limited TTS quality.

Mitigation:

- Evaluate multiple TTS solutions
 - Use appropriate language-specific models
 - Test with native speakers
-

Risk 4: High Voice Latency

Multiple AI services can introduce delays.

Mitigation:

- Streaming
 - Caching
 - Smaller models
 - Async processing
 - Efficient retrieval
-

Risk 5: False Warnings

Incorrect warnings may reduce farmer trust.

Mitigation:

- Conservative thresholds
- Source validation
- Severity rules
- Agricultural expert review

- Warning evaluation
-

Risk 6: Limited Local Agricultural Data

The RAG system may initially have limited Ethiopian agricultural information.

Mitigation:

- Prioritize trusted Ethiopian sources
 - Build metadata-rich documents
 - Continuously improve the knowledge base
 - Evaluate retrieval quality
-

Risk 7: Free-Tier API Limits

Free AI, weather, or messaging services may impose quotas.

Mitigation:

- Use provider abstraction
 - Cache responses
 - Keep mock implementations
 - Support multiple providers
 - Use open-source/local models where practical
-

70. Future Production Architecture

The MVP architecture is intentionally simpler.

A future production system could become:

Telecom Network

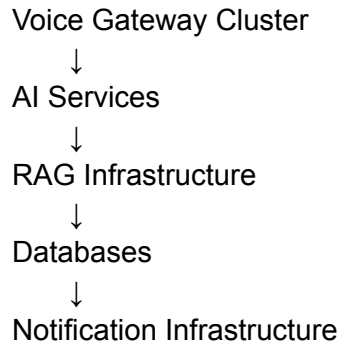


SIP / Telecom Gateway



Load Balancer





The same Hello Farmer AI layer could later operate alongside existing agricultural telephone infrastructure, including 8028.

Production integrations could include:

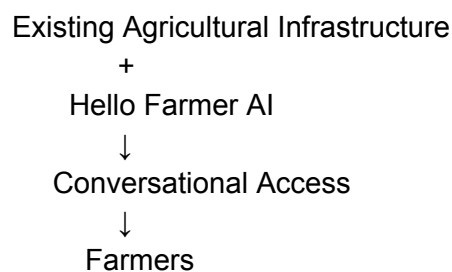
- Telecom SIP trunks
- Short codes
- Production SMS providers
- Government agricultural systems
- Weather providers
- Agricultural data providers
- Extension systems

These are future integrations, not MVP requirements.

71. Partnership and Deployment Model

Hello Farmer can eventually operate as an AI intelligence layer connected to existing agricultural information infrastructure.

Conceptually:



This allows existing infrastructure to retain its role while Hello Farmer improves accessibility and conversational interaction.

The long-term goal is interoperability rather than building an isolated agricultural information system.

72. Expected Impact

Hello Farmer aims to improve agricultural information access by:

- Reducing menu navigation
- Reducing keypad interaction
- Reducing first-time caller friction
- Improving access for low-literacy users
- Supporting local languages
- Providing personalized agricultural guidance
- Connecting advice with weather information
- Providing post-call reminders
- Sending proactive agricultural warnings
- Extending agricultural information access to basic phones
- Supporting smartphone users through a richer web experience

The expected impact should be validated through pilot data rather than assumed.

73. Key Differentiator

Hello Farmer combines:

Existing Telephone Access

+

Zero-Keypress Conversational AI

+

Five Ethiopian Languages

+

Agricultural RAG

+

Dynamic Farmer Context

+

Weather Intelligence

+

Proactive Warnings

+

Localized SMS

+

Web Experience

The individual technologies are not the main innovation.

The key innovation is how they are combined around the farmer's actual interaction.

The farmer does not need to understand:

- IVR menus
- Agricultural information categories
- Technical terminology
- Complex registration procedures

The farmer simply explains the problem.

The AI handles the complexity behind the scenes.

74. Core Product Principle

The entire project is built around one principle:

Do not make the farmer learn how to use the agricultural system. Make the agricultural system learn how to understand the farmer.

This changes agricultural information access from:

System → Farmer

to:

Farmer ↔ AI

The farmer speaks naturally.

The system understands.

The system retrieves trusted knowledge.

The system considers context.

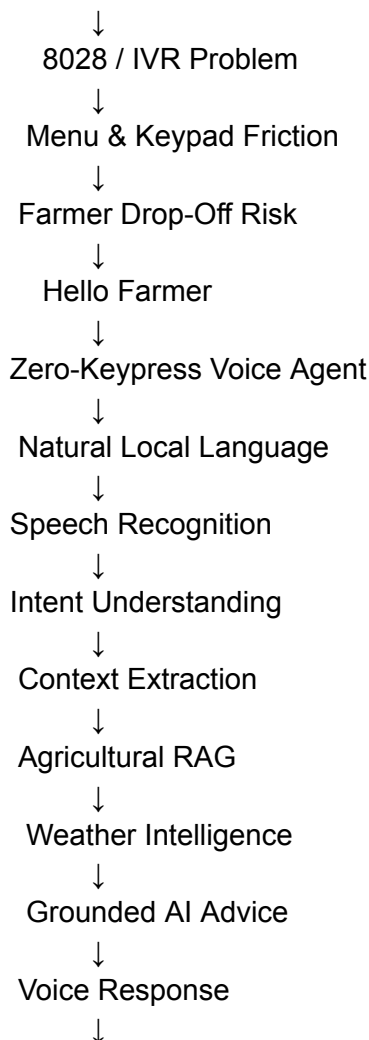
The system provides advice.

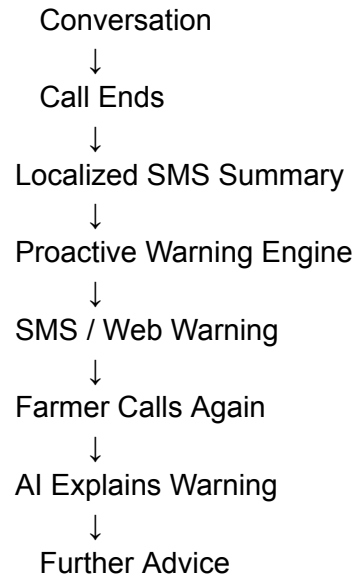
The system can warn the farmer before a problem becomes severe.

75. Complete Hello Farmer Interaction

The complete experience can be summarized as:

Existing Agricultural Telephone Access





This creates a continuous agricultural intelligence loop:

Detect → Warn → Explain → Advise

76. MVP Demonstration Scenario

A strong MVP demonstration can use the following scenario.

Step 1

A farmer uses a SIP softphone to call the Hello Farmer test extension.

Step 2

Asterisk answers the call.

Step 3

Hello Farmer asks:

"How can I help you with your farming today?"

Step 4

The farmer speaks in one of the supported languages.

For example:

"My maize leaves are turning yellow after the rain."

Step 5

The system detects:

Language: Amharic

Crop: Maize

Problem: Yellow leaves

Weather Context: Recent rain

Intent: Crop Problem

Step 6

The RAG system retrieves relevant maize agricultural information.

Step 7

The weather service provides current/recent weather context.

Step 8

The LLM generates grounded guidance.

Step 9

The response is converted to speech.

Step 10

The farmer hears the response.

Step 11

The farmer interrupts:

"What should I check first?"

The AI stops speaking and responds to the new question.

Step 12

The farmer ends the call.

Step 13

The system generates a concise localized SMS summary.

Step 14

The warning engine continues monitoring relevant agricultural risks.

This demonstrates the complete Hello Farmer concept without requiring production telecom infrastructure.

77. Final MVP Definition

The Hello Farmer MVP is not intended to be a complete national agricultural platform.

It is a working demonstration of a conversational agricultural intelligence system.

The MVP proves that:

1. A farmer can access agricultural AI through a telephone call.
 2. The farmer can speak naturally without navigating DTMF menus.
 3. The system can understand multiple Ethiopian languages.
 4. The AI can identify agricultural intent.
 5. The AI can extract relevant farmer context.
 6. The system can retrieve trusted agricultural knowledge.
 7. Weather information can be incorporated into relevant answers.
 8. The AI can provide grounded agricultural guidance.
 9. The farmer can interrupt the AI naturally.
 10. The system can generate a localized post-call summary.
 11. The system can generate proactive agricultural warnings.
 12. Warnings can be delivered through SMS and web notifications.
 13. Smartphone users can access a richer web experience.
 14. The complete system can run using Docker and free/open-source infrastructure.
 15. Production telecom integration can be added later without redesigning the core AI architecture.
-

78. Conclusion

Hello Farmer addresses a fundamental accessibility problem in agricultural information services.

Ethiopia already has telephone-based agricultural information infrastructure. The challenge is making that infrastructure easier and more natural for farmers to use.

Traditional IVR systems require farmers to adapt to menus, keypad sequences, registration procedures, and rigid information categories.

Hello Farmer takes a different approach.

The farmer speaks naturally.

The AI understands the language.

The system identifies the agricultural problem.

Farmer context is extracted dynamically.

Trusted agricultural information is retrieved.

Weather information is considered when relevant.

The AI generates grounded guidance.

The response is delivered through voice.

After the call, the farmer can receive a localized summary.

When an agricultural risk is detected, the system can proactively send a warning.

For smartphone users, the same intelligence can be accessed through a web experience.

For the MVP, Asterisk and Docker provide the telephone infrastructure without requiring a telecom provider or production 8028 integration. Free-tier services and open-source components are used wherever practical so that the complete concept can be developed and demonstrated with minimal cost.

The long-term vision is to create an agricultural information system that does not force farmers to learn complicated technology.

Instead, the technology learns to understand the farmer.

Hello Farmer makes agricultural information conversational, accessible, multilingual, contextual, and proactive.